BixoQuest: Desenvolvimento do Model de um RPG de Simulação Universitária em Java

Glauber Eduardo S. Silva, Gabriel Coelho

Departamento de Ciências Exatas – Universidade Estadual de Feira de Santana (UEFS), Av. Transnordestina, s/n – Novo Horizonte – 44.036-900 – Feira de Santana – BA – Brasil

glauberdudu71@gmail.com, gabriel4141.a@gmail.com

Resumo. *Este artigo apresenta o desenvolvimento da Fase 1 do jogo BixoQuest, um RPG de simulação da vida universitária implementado em Java. O objetivo desta etapa foi modelar a camada Model do sistema seguindo o padrão MVC, aplicando os princípios da Programação Orientada a Objetos. A metodologia adotada incluiu a diagramação UML das classes, a implementação das regras de negócio e a validação por meio de testes de unidade com JUnit 5. Os resultados demonstram uma base sólida e extensível para as fases seguintes do projeto.*

1. Introdução

O ingresso no ensino superior representa um período de transição desafiador, especialmente em cursos com alta carga de exigência técnica e analítica. Na Universidade Estadual de Feira de Santana (UEFS), o cotidiano de um estudante calouro de Engenharia de Computação vai muito além da simples absorção de conteúdos curriculares em sala de aula. O ambiente universitário impõe o desenvolvimento rápido de habilidades de autogestão, exigindo que o aluno concilie, de forma simultânea e equilibrada, o seu desempenho acadêmico com a preservação de sua saúde, o controle de suas finanças e a manutenção de sua motivação ao longo dos semestres. Essa intrincada teia de responsabilidades e variáveis cria um cenário complexo, onde cada decisão diária impacta diretamente a permanência e o sucesso do discente na instituição acadêmica.

Diante dessa realidade, surge a proposta do *BixoQuest*, um jogo de interpretação de papéis (RPG) de simulação projetado para modelar computacionalmente essa jornada universitária. O problema técnico primordial, proposto e desenvolvido no contexto da disciplina EXA863 (MI - Programação) da UEFS, consiste em traduzir toda a dinamicidade e a complexidade da vida de um estudante para um software estruturado, modular e escalável. O desafio reside em aplicar os paradigmas de Programação Orientada a Objetos (POO) e as boas práticas de Engenharia de Software para criar uma representação fiel das variáveis humanas e das interações do campus, garantindo que o núcleo do sistema seja seguro e livre de comportamentos anômalos.

Nesse contexto, o objetivo geral deste trabalho é desenvolver a base lógica e o motor central de regras do jogo *BixoQuest*. Para assegurar a qualidade e a fundação estrutural do software, estabeleceram-se como objetivos específicos para a Fase 1 do projeto: (i) diagramar a arquitetura das entidades do sistema utilizando a Linguagem de Modelagem Unificada (UML); (ii) implementar a camada *Model* de forma completa e

encapsulada, isolando a lógica de negócio de futuras interfaces gráficas; e (iii) validar a consistência matemática e a segurança das regras do jogo através de uma suíte abrangente de testes de unidade automatizados, construída com o *framework* JUnit 5.

Este documento está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica; a Seção 3 detalha a metodologia adotada; a Seção 4 discute os resultados obtidos; e a Seção 5 apresenta as conclusões e perspectivas futuras.

2. Fundamentação Teórica

A construção de um *software* de simulação requer uma arquitetura sólida e fundamentada em princípios de engenharia de *software* comprovados pela literatura acadêmica e pela prática da indústria. Esta seção apresenta os conceitos técnicos que embasam o desenvolvimento do núcleo lógico deste projeto, abordando o paradigma de Programação Orientada a Objetos, o padrão arquitetural *Model-View-Controller* (MVC) e a disciplina de testes de unidade.

2.1. Programação Orientada a Objetos (POO)

A Programação Orientada a Objetos (POO) é um paradigma de desenvolvimento de *software* que busca estruturar os sistemas computacionais de maneira análoga à organização do mundo real. Diferentemente da programação procedural, que separa dados e funções, a POO encapsula ambos dentro de entidades coesas chamadas "objetos" (Deitel & Deitel, 2016). Um objeto é composto por um estado interno (atributos) e um comportamento associado (métodos), permitindo a modelagem de sistemas complexos através da colaboração entre essas instâncias. Segundo Larman (2007), a adoção deste paradigma facilita a compreensão do domínio do problema e promove a criação de *softwares* mais fáceis de manter e estender.

Um dos pilares fundamentais da POO é o Encapsulamento, princípio que preconiza a ocultação dos detalhes de implementação e a proteção dos dados internos de um objeto contra acessos diretos e não autorizados. A interação com o estado do objeto deve ocorrer estritamente através de uma interface pública bem definida, geralmente composta por métodos modificadores e acessores. De acordo com Deitel e Deitel (2016), o encapsulamento é vital para garantir a integridade dos dados, permitindo a implementação de estratégias de "programação defensiva" onde o objeto valida e filtra as informações antes de alterar seu próprio estado.

Além do encapsulamento, a Herança e o Polimorfismo são mecanismos cruciais para a reutilização de código e a flexibilidade do *design*. A herança permite que novas classes sejam criadas a partir de classes existentes, herdando seus atributos e comportamentos e adicionando especificidades. O polimorfismo, por sua vez, complementa essa hierarquia ao permitir que objetos de tipos distintos — mas que compartilham uma mesma classe base ou interface — sejam tratados de maneira uniforme pelo sistema (Larman, 2007). Essa capacidade permite que o sistema invoque comportamentos generalizados, enquanto a execução real se adapta dinamicamente à classe específica do objeto em tempo de execução.

2.2. Padrão Arquitetural MVC (*Model-View-Controller*)

O padrão arquitetural *Model-View-Controller* (MVC) foi projetado para promover a separação de interesses (*separation of concerns*) em sistemas interativos, decompondo a aplicação em três componentes lógicos principais: o Modelo (*Model*), a Visão (*View*) e o Controlador (*Controller*). O objetivo central desta arquitetura é isolar a interface com o usuário das regras de negócio e da manipulação de dados, permitindo que cada camada evolua de forma independente (Sommerville, 2011).

No contexto do MVC, o *Model* constitui o coração do sistema. Ele encapsula o estado da aplicação, as regras de negócio intrínsecas e a lógica de processamento dos dados. De acordo com Gamma et al. (1995), o *Model* deve ser completamente cego e independente de como essas informações serão formatadas ou apresentadas visualmente ao usuário. Devido a essa responsabilidade central, durante a fase embrionária do desenvolvimento (Fase 1), o esforço de engenharia é tradicionalmente concentrado de forma exclusiva na construção e validação robusta deste componente, antes da implementação de qualquer interface gráfica ou controle de interação complexo.

2.3. Testes de Unidade

A prática de Testes de Unidade consiste na validação isolada e automatizada da menor parte testável de uma aplicação, como métodos individuais ou classes específicas. O objetivo é assegurar que cada componente do *software* funcione estritamente conforme projetado, independentemente do restante do sistema. Segundo Martin (2009), um código testado unitariamente garante previsibilidade e reduz o acoplamento, uma vez que o isolamento necessário para o teste força o desenvolvedor a criar *designs* mais coesos e desacoplados.

A adoção sistemática de testes de unidade oferece benefícios que transcendem a simples detecção de *bugs*. Conforme destaca Beck (2002), suítes de testes bem escritas servem como uma "documentação viva" do comportamento esperado do sistema, facilitam refatorações agressivas e previnem regressões matemáticas e lógicas durante a evolução do *software*. Para estruturar esses testes de forma clara e legível, a indústria consolidou o uso do padrão AAA (*Arrange, Act, Assert*), que divide o teste em três etapas: preparação do cenário (instanciação e configuração), execução da ação a ser validada e, por fim, a verificação (asserção) do resultado obtido contra o resultado esperado (Martin, 2009).

3. Metodologia

Para nortear a modelagem, os seguintes requisitos e funcionalidades centrais foram estabelecidos para a Fase 1: o sistema deve permitir o gerenciamento de recursos vitais (energia, dinheiro, motivação, saúde); o jogador deve poder navegar entre sistemas de diálogo estruturado (Visual Novel) e menus de ações cotidianas (estilo *BitLife*); e o sistema deve gerar eventos acadêmicos aleatórios, conferindo dinamicidade à simulação.

3.1. Ambiente de Desenvolvimento e Ferramentas

O desenvolvimento do sistema foi realizado utilizando a linguagem Java, escolhida devido à sua maturidade, forte tipagem estática e suporte nativo e consolidado aos

paradigmas de Programação Orientada a Objetos. Como Ambiente de Desenvolvimento Integrado (IDE), utilizou-se o IntelliJ IDEA, que fornece ferramentas avançadas de refatoração e inspeção de código em tempo real. Para a garantia da qualidade e validação das regras de negócio, empregou-se o *framework* JUnit 5, selecionado por sua integração contínua com o ecossistema Java e sua eficiência na automação de testes unitários.

3.2. Arquitetura e Modelagem

A arquitetura do *software* foi concebida sob as diretrizes do padrão *Model-View-Controller* (MVC). No entanto, de acordo com o escopo delimitado para a Fase 1 do projeto, o esforço de engenharia foi direcionado exclusivamente para a concepção e implementação da camada *Model*.

Previamente à etapa de codificação, foi elaborado um Diagrama de Classes detalhado na Linguagem de Modelagem Unificada (UML). Este planejamento visual permitiu mapear as entidades principais, seus atributos e os relacionamentos necessários para simular o ecossistema universitário. O *Model* resultante foi programado para ser completamente agnóstico e isolado, garantindo que suas operações matemáticas e transições de estado ocorram sem qualquer dependência ou conhecimento prévio das futuras camadas de *View* ou *Controller*.

3.3. Implementação da Lógica

A transposição das regras do RPG para código foi pautada na aplicação rigorosa de técnicas de POO. Dentre as principais decisões de implementação, destacam-se três frentes arquiteturais:

- **Encapsulamento e Programação Defensiva:** A modelagem do protagonista do jogo foi centralizada na classe Jogador. Para assegurar a integridade da simulação, atributos vitais como saúde, energia e motivação foram fortemente encapsulados. O acesso a essas variáveis ocorre exclusivamente por meio de métodos mutadores projetados com lógica de programação defensiva. Esses métodos implementam funções de "clamp" (limitação), garantindo matematicamente que parâmetros como saúde e motivação não ultrapassem a faixa de 0 a 100, e a energia semanal permaneça no intervalo de 0 a 70, prevenindo falhas críticas ou estados inválidos durante o *gameplay*.
- **Abstração e Polimorfismo:** Para gerenciar as interações no campus de forma escalável, o comportamento dos personagens foi padronizado através da criação da interface NPC. Essa abstração obrigou a implementação de contratos universais, como o fornecimento de diálogos dinâmicos (getCenaAtual()) e a progressão de vínculos (aumentarRelacionamento()). Como resultado, as classes concretas Professor e Colega puderam ser tratadas de forma polimórfica pelo sistema. Isso permite que o mecanismo de jogo varra uma sala e interaja com qualquer personagem de maneira uniforme, sem a necessidade de lógicas condicionais atreladas ao tipo específico do objeto.
- **Gerenciamento de Estado com Enums:** A cronologia do jogo exige precisão no controle de turnos. Para mitigar vulnerabilidades associadas ao uso de "strings mágicas" na alocação temporal (como erros de digitação ao registrar

horários), a marcação de tempo foi modelada utilizando tipos enumerados, como o BlocoTempo e a SemanaEnum. Essa decisão proveu segurança estrita de tipagem em tempo de compilação, assegurando que o sistema manipule o cruzamento entre espaços geográficos (Locais) e a agenda dos personagens sem ambiguidades temporais.

- **Composição de Sistemas e Eventos Aleatórios:** Para simular a rotina além dos diálogos (mecânicas estilo *BitLife*), o projeto utilizou o princípio de Composição. A classe Jogador atua como um agregador central, instanciando menus modulares (Academico, Financeiro, Saude, Social). Para introduzir imprevisibilidade e fator *replay*, implementou-se a classe CriarRandom, funcionando como uma fábrica estática que injeta ações variáveis (eventos aleatórios) diretamente no fluxo dos menus, modificando os atributos da ação padrão.
-

3.4. Estratégia de Testes de Unidade

A validação da camada *Model* foi conduzida por meio de testes de unidade automatizados. Adotou-se a estratégia de mapeamento espelhado, onde cada classe de produção (ex: Jogador) recebeu uma classe correspondente no diretório de testes (JogadorTest), garantindo o isolamento da análise.

A escrita dos testes seguiu o padrão AAA (*Arrange, Act, Assert*). O foco das verificações concentrou-se no estresse dos limites dos atributos, no comportamento do sistema diante de exceções e fluxos atípicos, e na coesão da integração entre classes. Um exemplo dessa integração validada foi o processo onde o sistema converte uma resposta estática (OpcaoResposta) em um evento transitivo (AcaoDialogo), aplicando com sucesso os impactos matemáticos previstos diretamente nos atributos da classe Jogador.

3.5. Uso de Inteligência Artificial Generativa

Conforme as diretrizes da disciplina para adoção de ferramentas de auxílio, declara-se a utilização de plataformas de Inteligência Artificial Generativa (IAG) como recurso de suporte durante o desenvolvimento desta fase. A IAG foi empregada especificamente para:

- Debater e explorar alternativas arquiteturais para a estrutura de classes (como o *design pattern* Factory para os diálogos e a interface genérica para *NPCs*).
- Fornecer apoio na depuração (*debugging*) de fluxos lógicos e gargalos de execução.
- Aprofundar explicações conceituais referentes aos pilares da POO e à responsabilidade estrita do *Model* no padrão MVC.
- Auxiliar na estruturação sintática e organização tópica da presente documentação técnica.

Ressalta-se que a IA atuou estritamente como agente facilitador e consultivo. Todo o código gerado, os testes concebidos e as descrições elaboradas foram minuciosamente revisados, adaptados ao escopo do *BixoQuest* e compreendidos em sua

totalidade pelos desenvolvedores do projeto.

4. Resultados e Discussão

A conclusão da Fase 1 do *BixoQuest* resultou em um núcleo lógico robusto, onde as regras de negócio foram implementadas com foco em extensibilidade e integridade de dados. Esta seção detalha a estrutura produzida, as justificativas para as escolhas de *design* e a validação das funcionalidades por meio de testes automatizados.

4.1. Visão Geral do Model

O pacote model foi estruturado para refletir com precisão os componentes de um RPG de simulação. A arquitetura é composta por 16 classes e uma interface principal, organizadas em hierarquias que priorizam o reaproveitamento de código. A classe Jogador centraliza o estado do sistema, enquanto a infraestrutura de interação é sustentada pelas classes Local, CenaDialogo e OpcaoResposta. O ecossistema de personagens é regido pela interface NPC, implementada pelas classes Professor e Colega. A lógica de eventos e recompensas utiliza uma hierarquia de herança iniciada em Atributos, passando por Acao e culminando em AcaoDialogo, garantindo que qualquer impacto nos *status* do jogador siga um fluxo padronizado.

4.2. Decisões de Arquitetura e Justificativas

A implementação não se limitou a codificar requisitos, mas a aplicar padrões que facilitam a manutenção futura do projeto. As principais decisões tomadas foram:

- **Interface NPC para Polimorfismo:** A criação da interface NPC foi estratégica para reduzir o acoplamento. Ao definir métodos como getCenaAtual(), o sistema permitiu que o controlador interagisse com personagens tão distintos quanto o professor "Anfranserai" ou o colega "Suco" de forma idêntica. Essa decisão permite que novos tipos de personagens (como animais do campus) sejam adicionados sem alterar a lógica de navegação.
- **Herança de Atributos em Acao:** A classe Atributos foi estabelecida como base para a classe Acao para aplicar o princípio DRY (*Don't Repeat Yourself*). Como todas as ações do jogo (estudar, comer, interagir) resultam em alterações de saldo (saúde, dinheiro, desempenho), centralizar esses campos na classe mãe evitou a duplicação de atributos e simplificou o método somaAtributos().
- **Extensibilidade em OpcaoVisualNovel:** A classe OpcaoVisualNovel estende a classe Opcao original. Essa herança preservou a compatibilidade com o sistema de menus genérico, ao mesmo tempo em que permitiu a inclusão de atributos específicos para a narrativa, como o nomeLocalReal, necessário para vincular as escolhas do jogador ao mapa geográfico.
- **Navegação via String no SistemaDeRotas:** Optou-se por representar a localização do jogador no menu através de uma *String* denominada pag (ex: "021"). O uso de concatenação e *substring* para avançar ou retroceder níveis de

menu provou-se mais simples e menos propenso a erros de memória do que a implementação de uma pilha (*Stack*) de objetos complexos, mantendo a clareza hierárquica.

- **Hierarquia do Sistema de Menus:** A criação da classe mãe Menu, da qual herdam Academico, Financeiro, Saude e Social, foi uma decisão tomada para aplicar o reaproveitamento máximo de lógica. Como todos os menus do jogo lidam com a renderização de opções, consumo de escolhas e acúmulo temporário de atributos, centralizar essa lógica na classe Menu eliminou a replicação de código de controle de interface no *Model*.
- **A Classe Abstrata Personagem e Colega:** Com a expansão do sistema social, criou-se a classe abstrata Personagem, da qual Colega herda. Essa decisão permitiu a incorporação de novos atributos mecânicos (como bônus em disciplinas baseado no "cargo" do colega, ex: monitor de Cálculo), conectando a interação social diretamente ao progresso acadêmico da simulação de forma limpa.

4.3. Cobertura de Testes e Validação dos Requisitos

Para assegurar que o *Model* atende às exigências do edital do problema, foi desenvolvida uma suíte de testes que valida cada requisito funcional. A bateria de testes contou com a execução de diversos casos de teste de unidade individuais, alcançando uma taxa de sucesso de 100%. A cobertura de código medida pela ferramenta nativa da IDE IntelliJ indicou uma cobertura de 100% das classes do pacote Model. A tabela abaixo resume a correlação entre os requisitos e os testes executados:

Tabela 01. Contém os requisitos do projeto na parte "*Visual Novel*"

Requisito do PBL / Domínio	Classe Testada	O que foi validado pelo JUnit 5
Atributos do personagem	JogadorTest	Travas de segurança (clamp) para energia, saúde e motivação (limites de 0 a 100) e formatação monetária.
Lógica de Herança e Saldos	AtributosTest, AcaoTest	Independência de objetos (construtor de cópia), soma de atributos matemáticos e prevenção de referências vazadas.
Conversão de Escolhas	OpcaoRespostaTest	Conversão correta de respostas textuais em ações sistêmicas, roteando os pontos de conhecimento para as disciplinas exatas (EXA, TEC, ALG).
Ações com impacto	AcaoDialogoTest	O método aplicarJogador() modifica os atributos do protagonista corretamente sem interferência entre diferentes status.
Personagens e diálogos	ProfessorTest, ColegaTest	Progressão correta do nível de relacionamento e a troca automática das cenas de diálogo de acordo com o vínculo.
Inicialização do Mundo	ConstrutorDeMapaTest	Geração do mapa com integração impecável entre os nomes reais dos locais e as chaves de busca do dicionário (HashMap).
Locais com	LocalTest	Registro e verificação de presença correta dos NPCs nas

NPCs		salas específicas de acordo com o turno (BlocoTempo).
Navegação por menus	SistemaDeRotasTest	Concatenação de rotas via String, funcionamento do botão "voltar" e travamento em menus de ações críticas.
Mecânicas de Simulação	MenuTest, CriarRandomTest	Geração correta de eventos aleatórios e o acúmulo de atributos temporários antes de transferi-los efetivamente ao Jogador.

Durante o processo de desenvolvimento e execução dos testes, erros de lógica significativos puderam ser mapeados e corrigidos precocemente. Por exemplo, identificou-se que a acumulação sucessiva de ações no sistema de Menus poderia levar o saldo financeiro a valores negativos, e que índices fora do escopo (Out of Bounds) causavam falhas na fábrica CriarRandom. Os testes permitiram calibrar a função clamp e os validadores de limites para impedir que essas anomalias chegassem à renderização visual do jogo.

Os testes confirmaram que comportamentos indesejados, como a saúde tornar-se negativa após uma interação hostil ou o dinheiro ultrapassar limites não previstos, são impossíveis de ocorrer na lógica atual, garantindo uma base estável para a Fase 2.

4.4. Diagrama de Classes

O Diagrama de Classes completo, em conformidade com as normas UML, encontra-se disponível no anexo deste relatório. Em termos estruturais, o diagrama ilustra uma forte relação de **composição** entre a classe Local e a lista de NPCs vinculada à agenda. Nota-se também o uso de **implementação de interface** por parte dos personagens e a **associação** entre Jogador e o histórico de Ações. Essas relações garantem que o sistema seja modular: é possível alterar a lógica de um diálogo específico sem afetar o cálculo de desempenho acadêmico, evidenciando a alta coesão do modelo desenvolvido.

5. Conclusões

5.1. Veredito

A conclusão da Fase 1 do projeto *BixoQuest* confirma que os objetivos lógicos iniciais foram atingidos. Desenvolveu-se a camada *Model* de forma robusta e encapsulada. Como **objetivo extra** (além do escopo obrigatório), o projeto inovou ao mesclar duas arquiteturas de *gameplay* distintas: um sistema de locomoção e diálogos textuais (Visual Novel) integrado a um sistema de gerenciamento de tempo e ações estáticas (BitLife). É importante ressaltar que os **objetivos não cumpridos** nesta entrega, especificamente a persistência de dados (salvamento de progresso) e a interface gráfica, não configuram falha, pois são escopos deliberadamente reservados para as Fases 2 e 3 do cronograma da disciplina.

5.2. Pontos Positivos e Negativos

A avaliação crítica do desenvolvimento revelou pontos de destaque e oportunidades de melhoria. Como ponto positivo, a opção pela interface NPC conferiu ao código uma alta capacidade de extensibilidade, permitindo que novos comportamentos sejam

adicionados sem impactar as classes existentes. Além disso, a implementação de programação defensiva na classe Jogador eliminou a ocorrência de estados inválidos, garantindo a consistência dos atributos durante o fluxo de jogo. O mapeamento espelhado dos testes de unidade facilitou a identificação de falhas de forma isolada, acelerando o ciclo de correção.

Por outro lado, observaram-se desafios significativos. A curva de aprendizado inicial para o domínio das ferramentas do JUnit 5 exigiu um investimento de tempo superior ao planejado. Adicionalmente, o desenvolvimento do modelo de forma puramente abstrata, sem uma interface visual imediata, dificultou a percepção estética e o ajuste fino do ritmo das interações narrativas. Por fim, a hierarquia de classes estabelecida entre Atributos, Acao e AcaoDialogo, embora tecnicamente eficiente para o reaproveitamento de código, resultou em construtores com elevada quantidade de parâmetros, aumentando a complexidade de manutenção dessas instâncias específicas.

5.3. Lições Aprendidas

O processo de engenharia reforçou a premissa de que o planejamento precede a eficiência. A elaboração detalhada do Diagrama de Classes UML antes do início da codificação provou ser um diferencial, evitando retrabalhos estruturais significativos durante a implementação lógica. No campo técnico, a prática sistemática de validar limites matemáticos (piso e teto) via programação defensiva e testes automatizados demonstrou ser a estratégia mais eficaz para evitar *bugs* silenciosos em sistemas que dependem de múltiplas variáveis interconectadas.

5.4. Trabalhos Futuros

As etapas subsequentes do projeto darão continuidade à evolução do software conforme o cronograma estabelecido. A Fase 2 **focará** na implementação da persistência de dados, permitindo que o progresso do estudante seja salvo e carregado entre sessões. Na Fase 3, o sistema receberá sua interface gráfica definitiva utilizando a biblioteca JavaFX. A arquitetura modular construída nesta primeira etapa foi projetada especificamente para suportar essas expansões, garantindo que a inserção de novas camadas não comprometa a integridade do *Model* já validado.

Referências

BECK, K. *Test-Driven Development: By Example*. Boston: Addison-Wesley, 2002.

DEITEL, P.; DEITEL, H. *Java: Como Programar*. 10. ed. São Paulo: Pearson Education do Brasil, 2016.

GAMMA, E. et al. *Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos*. Porto Alegre: Bookman, 2000.

GOOGLE. *Gemini*. Modelo de Inteligência Artificial Generativa. Disponível em: https://gemini.google.com. Acesso em: 12 abr. 2026.

JUNIT TEAM. *JUnit 5 User Guide*. Disponível em: https://junit.org/junit5/docs/current/user-guide/. Acesso em: 12 abr. 2026.

LARMAN, C. *Utilizando UML e Padrões: Uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo*. 3. ed. Porto Alegre: Bookman, 2007.

MARTIN, R. C. *Código Limpo: Habilidades Práticas do Agile Software*. Rio de Janeiro: Alta Books, 2009.

SOMMERVILLE, I. *Engenharia de Software*. 9. ed. São Paulo: Pearson Prentice Hall, 2011.